

第五章—中间代码生成

编译程序的目标是将源程序翻译成等价的目标程序，为了得到高效目标代码，有时会先生成中间代码。将高级语言翻译成中间代码需要完成哪些工作呢？



- 1 中间代码的形式
- 2 属性文法
- 3 语法制导翻译技术



回顾

语法单位 (输入)

中间代码 (要翻译的结果)

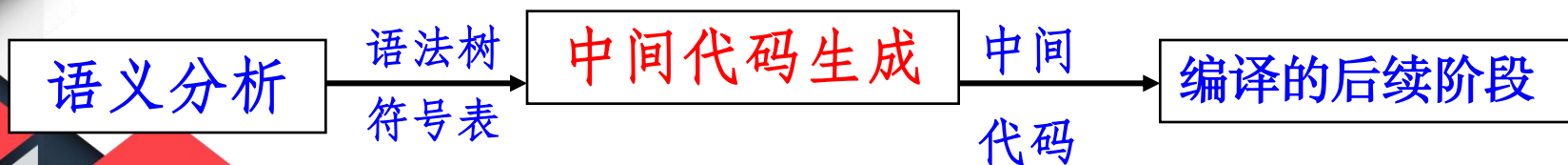
属性文法 (语义规则的表示方法)

语法制导翻译技术 (翻译的方法)

回顾

中间代码的几种形式

- 逆波兰
- 三地址代码 (三元式、四元式)
- 抽象语法树
- 字节码
- 有向无环图 (DAG)



生成中间代码是为了缩小高级语言和机器语言之间的鸿沟，可以设计多层中间语言。

回顾 属性文法

产生式

$L \rightarrow E \#$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语义规则

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$



回顾 属性文法

产生式

语义规则

$D \rightarrow TL$ $L.in := T.type$

$T \rightarrow int$ $T.type := integer$

$T \rightarrow real$ $T.type := real$

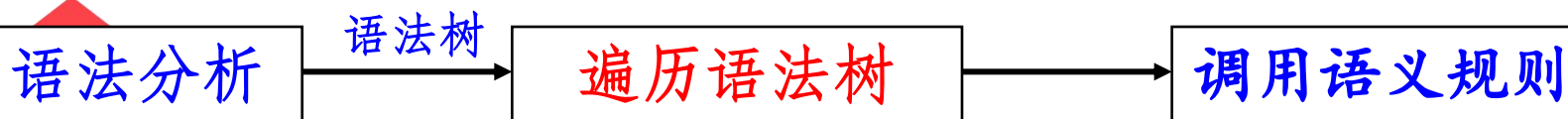
$L \rightarrow L_1, id$ $L_1.in := L.in$

$addtype(id.entry, L.in)$

$L \rightarrow id$ $addtype(id.entry,$
 $L.in)$

5.3.3 属性的计算顺序

基于属性文法的语义处理过程实际上就是属性的计算过程，首先对单词符号串进行语法分析，构造语法树，然后根据需要遍历语法树，在语法树的各结点处按照语义规则进行属性计算。



5.3.4 语法制导翻译的实现方法

- 语法制导翻译 (Syntax-Directed Translations) :
 - 一个句子的语义分析过程与语法分析过程同时进行。
 - 在文法中，文法符号有明确的意义，文法符号之间有确定的语义关系。属性描述语义信息，语义规则描述属性间的关系，将语义规则与语法规则相结合，在语法分析的过程中计算语义属性值。

5.3.4 语法制导翻译的实现方



法 • 常见的语法制导翻译类型:

- 语法分析采用递归下降分析方法时，利用**隐含堆栈**存储各递归子程序中的局部变量所表示的语义信息
- 语法分析采用LL(1)分析法时，利用**翻译文法**进行语法制导的翻译。翻译文法是在描述语言的**文法**中加入**语义动作**的符号。
- 语法分析采用自下而上方法时，使用与语法分析栈同步操作的**语义栈**进行语法制导翻译。
- S-属性文法和L-属性文法翻译。属性文法的文法符号和动作符号都带有语义属性和同一产生式中各属性间的运算规则。

5.3.4 语法制导翻译的实现方法

- 对应每一个产生式编制一个语义子程序（语义规则），在进行语法分析的过程中，当一个产生式获得匹配时，就调用相应的语义子程序实现语义检查与翻译。即在自下而上语法分析的过程中，每一次归约时就调用相应的语义子程序，在自上而下的语法分析过程中每一次推导就调用相应的语义子程序。
- 在产生式的右部的适当位置，插入相应的语义动作，按照分析的进程，执行遇到的语义动作。

5.3.4 语法制导翻译的实现方法

高级程序设计语言的语义子程序描述了一个产生式对应的翻译工作。

- 主要有：改变编译程序某些变量的值、查填各种符号表、发现并报告源程序错误、产生中间代码。
- 在描述语义动作时需要为每个文法符号赋以各种不同的语义值，如类型、地址、代码值等。
- 如果一个产生式中一个符号多次出现，就用上角标来区分，如： $E = E^{(1)} + E^{(2)}$

5.3.4 语法制导翻译的实现方法

❖ 语义子程序的一般形式

❖ 语义子程序写在该产生式后面的花括号内：

❖ (1) $X \rightarrow \dots$ { 语义子程序1 }

❖ (2) $Y \rightarrow \dots$ { 语义子程序2 }

❖ (3) $A \rightarrow XY$ { 语义子程序3 }

⌘ 例：台式计算器程序的语义子程序描述：

产生式	语义子程序
⌘ (0) $S' \rightarrow E$	{PRINT E. VAL}
⌘ (1) $E \rightarrow E^{(1)} + E^{(2)}$	{E. VAL = E ⁽¹⁾ . VAL + E ⁽²⁾ . VAL}
⌘ (2) $E \rightarrow E^{(1)} * E^{(2)}$	{E. VAL = E ⁽¹⁾ . VAL * E ⁽²⁾ . VAL}
⌘ (3) $E \rightarrow (E^{(1)})$	{E. VAL = E ⁽¹⁾ . VAL}
⌘ (4) $E \rightarrow i$	{E. VAL = LEXVAL}

5.3.4 语法制导翻译的实现方法

S-属性文法的自下而上翻译

X_n	$X_n.Val$	$\leftarrow \text{top}$
X_1	$X_1.Val$	
#		

语法栈 语义栈

- 增加语义栈
- 用属性表示文法符号 X 的语义信息。
- 语义信息与文法符号同时出栈和入栈

5.3.4 语法制导翻译的实现方法

例1：下面是一个算术表达式文法，每个产生式右边是它的语义动作，对输入串 $2 * 3 + 2$ 的规范归约的分析过程如下：

产生式	语义子程序
$E \rightarrow E^{(1)} + E^{(2)}$	$\{E \cdot V \text{ AL} = E^{(1)} \cdot V \text{ AL} + {}^i E^{(2)} \cdot V \text{ AL}\}$
$E \rightarrow E^{(1)} * E^{(2)}$	$\{E \cdot V \text{ AL} = E^{(1)} \cdot V \text{ AL} * {}^i E^{(2)} \cdot V \text{ AL}\}$
$E \rightarrow 0 1 2 \cdots 9$	$\{E \cdot V \text{ AL} = d d = 0, 1, \dots, 9\}$

5.3.4 语法制导翻译的实现方法

输入串
2 * 3 + 2 #
↑

—	#

(1) 初态
语法树为空

* 3 + 2 #
↑

	2
	#

(2) 移进 2
语法树为空

* 3 + 2 #
↑

E • VAL = 2	E
—	#

(3) 归约 2
E(2)
|
2

3 + 2 #
↑

* i	*
E • VAL = 2	E
—	#

(4) 移进 *
E(2)
|
2

5.3.4 语法制导翻译的实现方

输入串 $+2\#$
↑

	3
$\star i$	$\star$
$E \cdot VAL = 2$	E
—	#

(5) 移进 3
E(2)

语法树
|
2 $\star$ 3

$+2\#$
↑

$E \cdot VAL = 3$	E
$\star i$	$\star$
$E \cdot VAL = 2$	E
—	#

(6) 归约 3
E(2) E(3)

|
2 $\star$ 3

$+2\#$
↑

$E \cdot VAL = 6$	E
—	#

(7) 归约 $E \star E$

E(6)
/ | \
E(2) $\star$ E(3)
| | |
2 $\star$ 3

$2\#$
↑

$+i$	+
$E \cdot VAL = 6$	E
—	#

(8) 移进 +
E(6)

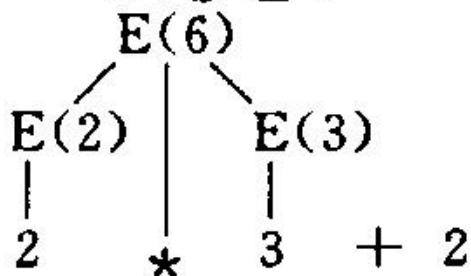
/ | \
E(2) $\star$ E(3) +
| | |
2 $\star$ 3

5.3.4 语法制导翻译的实现方法

↑

	2
+ ⁱ	+
E·VAL=6	E
—	#

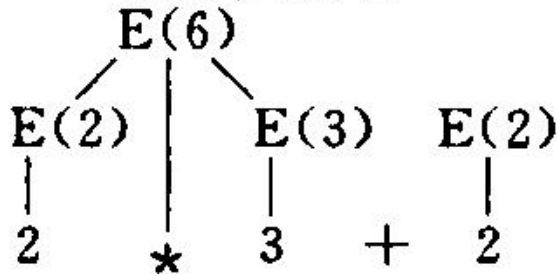
(9) 移进 2



↑

E·VAL=2	E
+ ⁱ	+
E·VAL=6	E
—	#

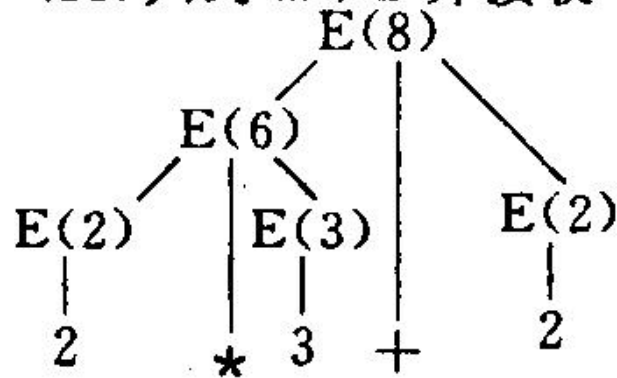
(10) 归约 2



↑

E·VAL=8	E
—	#

(11) 归约 E+E 并接收



 河南师范大学
HENAN NORMAL UNIVERSITY

$$\begin{aligned} (1) \quad E &\rightarrow E + T \mid T \\ (2) \quad T &\rightarrow T * F \mid F \\ (3) \quad F &\rightarrow (E) \mid i \end{aligned}$$
[illegible]



序号	状态栈	符号栈	语义栈	归约产生式	输入串
1	0	#	-		3*5+4#
2	05	#3	--		*5+4#
3	03	#F	-3	$F \rightarrow i$	*5+4#
4	02	#T	-3	$T \rightarrow F$	*5+4#
5	027	#T*	-3-		5+4#
6	0275	#T*5	-3--		+4#
7	0271 <u>0</u>	#T*F	-3-5	$F \rightarrow i$	+4#
8	02	#T	-15	$T \rightarrow T*F$	+4#
9	01	#E	-15	$E \rightarrow T$	+4#
10	016	#E+	-15-		4#
11	0165	#E+4	-15--		#
12	0163	#E+F	-15-4	$F \rightarrow i$	#
13	0169	#E+T	-15-4	$T \rightarrow F$	#
14	01	#E	-19	$E \rightarrow E+T$	#
15				接受	

5.3.4 语法制导翻译的实现方法

翻译过程的特点：

- 句柄归约在先，语义动作在后。
- 归约时，栈内句柄各符号的语义信息与该句柄同时出栈，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量同时入栈。
- 为了在某处调用语义动作，就必须先归约，因此，有时需要改写文法。

5.4 常见语句的语法制导的翻译



- 高级语言的语句分类：
 - 说明语句——定义各种名字的属性
 - 可执行语句——用于完成指定的功能，涉及到指令代码
- 语义翻译也分两类：
 - 翻译说明语句时，将所定义的名字的各种属性登记到符号表中
 - 翻译可执行语句时，首先应根据各源语句的语法结构和语义设计出它的目标代码结构，找出源与目标的对应关系，给出对信息(数据表示)描述和从源到目标的变换算法。语义子程序则根据变换方法进行翻译。

5.4.1 声明语句的语义处理

声明语句的作用：

- 就是说明类型等属性信息，在翻译时主要是填符号表

声明语句分类：

- 变量声明语句
- 常量声明语句
- 函数声明语句

一 语义变量和语义函数

涉及到的语义变量和函数

- **X.PLACE**: 表示与X对应的变量在符号表中的位置。
- **ENTRY(id)**: 函数，查找变量id的入口地址；检查id.name是否在符号表中，如在则返回一指向该登录项的指针，否则返回NULL

二 常量声明语句

1. 常量说明语句的文法描述

- (1) $\langle \text{CONSTDCL} \rangle \rightarrow \text{const} \langle \text{CONSTDEF} \rangle$
- (2) $\rightarrow \varepsilon$
- (3) $\langle \text{CONSTDEF} \rangle \rightarrow \langle \text{CONSTDEF} \rangle; \text{id} = \text{INT}$
- (4) $\rightarrow \langle \text{CONSTDEF} \rangle; \text{id} = \text{STR}$
- (5) $\langle \text{CONSTDEF} \rangle \rightarrow \text{id} = \text{INT}$
- (6) $\rightarrow \text{id} = \text{STR}$

例: **const A = 123; K = 'ABC'**

2. 常量说明语句的翻译

策略: 先读完一个说明语句再翻译。

在归约时, 执行语义动作, 将常量的值、类型、种属填入符号表。

3. 翻译的语义动作

产生式	语义处理
(5) <CONSTDEF> → id=INT	{ entry(id).val=entry(INT).val; entry(id).type=integer; entry(id).kind=数值常数; }
(6) <CONSTDEF> → id=STR	{ entry(id).val=entry(STR).val; entry(id).type=char; entry(id).kind=字符常数; }

将常

入口或值

所指符

将常数的类型填入
符号表的Type栏

将常数的种属填入
符号表的VAL栏

将常数的种属填入
符号表的Kind栏

(3) (4)产生式的翻译与(5)(6)产生式的翻译相同

(1)(2)产生式没有语义动作

const A=123; K= 'ABC'

Name		Type	Kind	Val	Addr
A	1	integer	数值常数	123	
K	1	char	字符常数	ABC	

三 变量声明语句

1. 变量说明语句的文法描述

- (1) $\langle \text{VARDCL} \rangle \rightarrow \text{var } \langle \text{IDS} \rangle \mid \epsilon$
- (2) $\langle \text{IDS} \rangle \rightarrow \text{id}, \langle \text{IDS} \rangle^{(1)}$
- (3) $\langle \text{IDS} \rangle \rightarrow \text{id:integer}$
- (4) $\langle \text{IDS} \rangle \rightarrow \text{id:char}$
- (5) $\langle \text{IDS} \rangle \rightarrow \text{id: bool}$
- (6) $\langle \text{IDS} \rangle \rightarrow \text{id: real}$

2. 变量说明语句的翻译

var exam, b, c: integer;

策略：读完一个说明语句，开始归约时，变量的类型朝前传递。

3. 翻译的语义动作

entry(id) 表示变量名id
在符号表中的入口

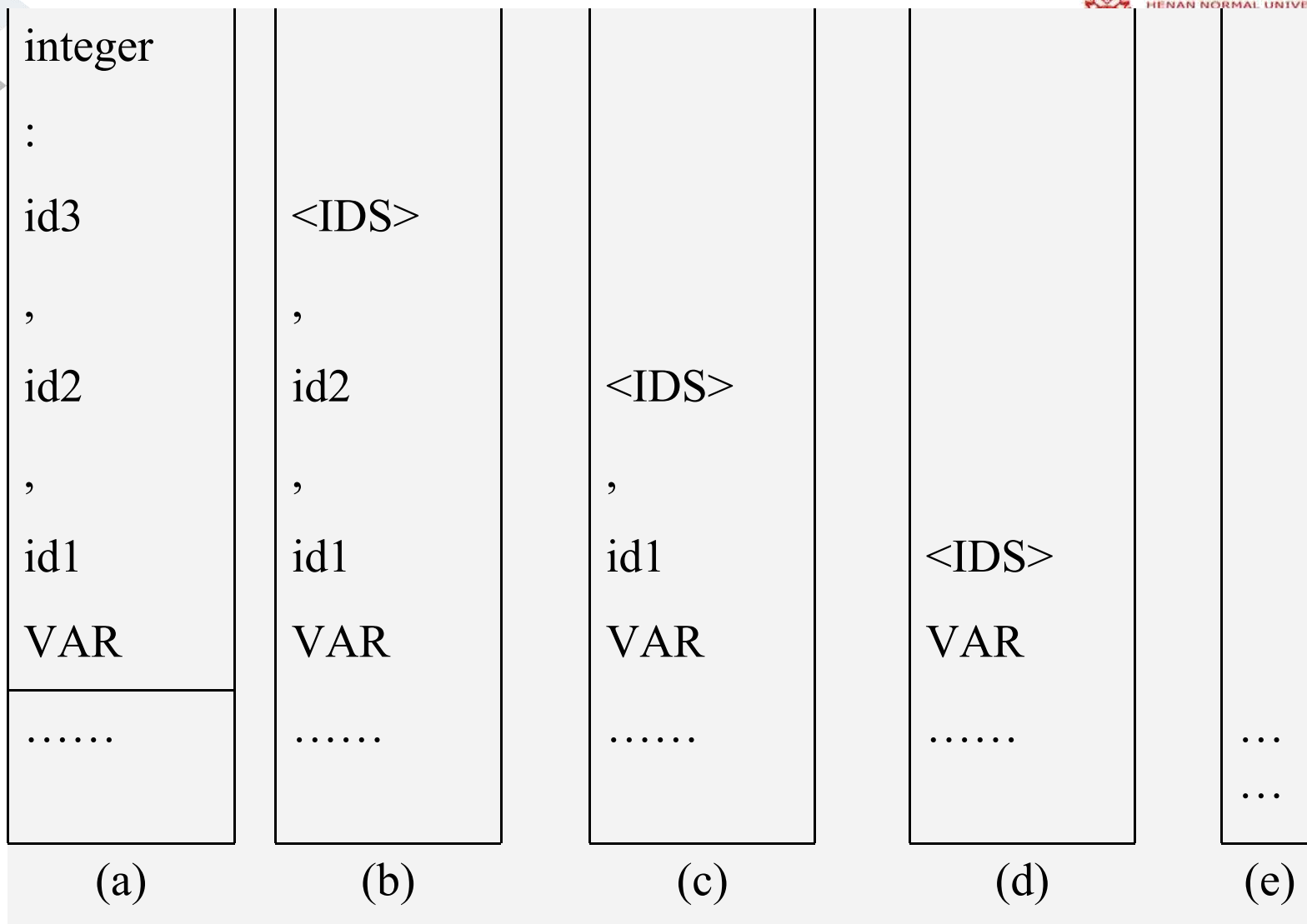
编号	产生式	语义动作
(4)	$\langle \text{IDS} \rangle \rightarrow \text{id}:\text{char}$	{ fill(entry(id),char); $\langle \text{IDS} \rangle \cdot \text{type} = \text{char}$ }
(3)	$\langle \text{IDS} \rangle \rightarrow \text{id}:\text{integer}$	{ fill(entry(id),integer); $\langle \text{IDS} \rangle \cdot \text{type} = \text{integer}$ }
(2)	$\langle \text{IDS} \rangle \rightarrow \text{id}, \langle \text{IDS} \rangle^{(1)}$	{ fill(entry(id), $\langle \text{IDS} \rangle^{(1)} \cdot \text{type}$); $\langle \text{IDS} \rangle \cdot \text{type} = \langle \text{IDS} \rangle^{(1)} \cdot \text{type}$ }
(1)	$\langle \text{VARDCL} \rangle \rightarrow \text{var } \langle \text{IDS} \rangle \varepsilon$	{ nop }

符号表:

var exam, b, c: integer;

Name		TYPE	KIND	VAL	ADDR
exam	4	integer			
b	1	integer			

VAR id1 , id2 , id3 : integer ; 的归约过程



四 函数声明语句

函数声明语句的语义：

- (1) 将函数名添加到符号表
- (2) 将函数的相关信息添加到符号表：函数返回值的类型、参数的个数、按照顺序每个参数的类型

小结

- 1 语法制导翻译技术
- 2 声明语句的语义处理
 - 常量声明语句
 - 变量声明语句
 - 函数声明语句



$S' \rightarrow S \quad \{\text{printf}(S.\text{val})\}$
 $S \rightarrow L^1.L^2 \quad \{S.\text{val} := L^1.\text{val} + L^2.\text{val} / 2^{L^2.\text{length}}\}$
 $S \rightarrow L \quad \{S.\text{val} := L.\text{val}\}$
 $L \rightarrow L^1B \quad \{L.\text{val} := L^1.\text{val} * 2 + B.\text{val},$
 $\quad L.\text{length} = L^1.\text{length} + 1 \quad \}$
 $L \rightarrow B \quad \{L.\text{val} := B.\text{val},$
 $\quad L.\text{length} := 1 \quad \}$
 $B \rightarrow 0 \quad \{B.\text{val} := 0\}$
 $B \rightarrow 1 \quad \{B.\text{val} := 1\}$

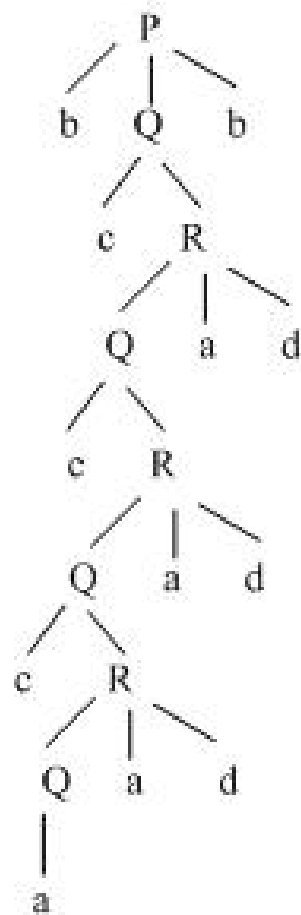


$S' \rightarrow S \quad \{\text{printf}(S.h)\}$
 $S \rightarrow a \quad \{S.h := 0\}$
 $S \rightarrow (L) \quad \{S.h := S.h + 1\}$
 $L \rightarrow L^{(1)}, S \quad \{S.h := L^{(1)}.h + S.h\}$
 $L \rightarrow S \quad \{L.h := S.h\}$



$P \rightarrow D \quad \{\text{printf}(D.h)\}$
 $D \rightarrow D^1; D^2 \quad \{D.h := D^1.h + D^2.h\}$
 $D \rightarrow \text{id}:T \quad \{D.h := 1\}$
 $D \rightarrow \text{proc id}; D^1; S \quad \{D.h := D^1.h + 1\}$





5. 解答:

$$(a+b)+(c+d/(e-3))*8$$

6. 解答:

(1) $ab-c+*$

(2) A not C D not or not or

(3)

$$abcde/+*+$$

(4) A B and C not D or or

(5) $a-bc-d+*+$

(6) A B or C D not E and or and



三元式	四元式
① $(+,a,b)$	1. $(+,a,b,T_1)$
② $(-,1,-)$	2. $(-,T_-, T_2)$
③ $(+,c,d)$	3. $(+,c,d,T_3)$
④ $(*,2,3)$	4. $(*, T_2, T_3, T_4)$
⑤ $(+,a,b)$	5. $(+,a,b,T_5)$
⑥ $(+,c,5)$	6. $(+, T_5, c, T_6)$
⑦ $(-,4,6)$	7. $(-, T_4, T_6, T_7)$



回顾

一个语义子程序描述了一个产生式对应的翻译工作。

- 主要有：改变编译程序某些变量的值、查填各种符号表、发现并报告源程序错误、产生中间代码。
- 在描述语义动作时需要为每个文法符号赋以各种不同的语义值，如类型、地址、代码值等。
- 如果一个产生式中一个符号多次出现，就用上角标来区分，如： $E = E^{(1)} + E^{(2)}$

回顾

• 语法制导翻译

- 一个句子的语义分析过程与语法分析过程同时进行。
- 在文法中，文法符号有明确的意义，文法符号之间有确定的语义关系。属性描述语义信息，语义规则描述属性间的关系，将语义规则与语法规则相结合，在语法分析的过程中计算语义属性值。

回顾

- 句柄归约在先，语义动作在后。
- 归约时，栈内句柄各符号的语义信息与该句柄同时出栈，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量同时入栈。
- 为了在某处调用语义动作，就必须先归约，因此，有时需要改写文法。



- 翻译说明语句时，将所定义的名字的各种属性登记到符号表中
- 翻译可执行语句时，首先应根据各源语句的语法结构和语义设计出它的**目标代码结构**，找出源与目标的对应关系，给出对信息(数据表示)描述和从源到目标的**变换算法**。语义子程序则根据变换方法进行翻译。

5.4.2 简单赋值语句的翻译

定义语义变量和函数

- GENCODE(OP, ARG1, ARG2, RESULT): 产生一个四元式，并填入四元式表，编号自动增1。
- NEWTEMP: 返回一个临时变量序号的函数。在翻译可执行语句的过程中，可能需要使用临时变量，假定用NEWTEMP过程来申请临时变量Ti，每申请一次，i增1。
- E.PLACE: 与非终结符E相联系的语义变量，可能是某变量的入口地址，或者为临时变量序号。
- NXQ: 即将要生成的四元式的编号。

1. 简单赋值语句的文法描述

例如: $x = 2 + 3 * 2;$

$$\begin{aligned} S &\rightarrow id = E \\ E &\rightarrow E1 + T \mid T \\ T &\rightarrow T1 * F \mid F \\ F &\rightarrow -F \mid id \mid (E) \end{aligned}$$

2. 简单赋值语句的代码结构

$id = aexpr;$

aexpr的代码

$id = aexpr.place$

3. 简单赋值语句的翻译 假定是整数运算



编号	产生式	语义规则
(1)	$S \rightarrow id = E$	{ gencode(=, E.PLACE, _, entry(id)) }
(2)	$E \rightarrow E_1 + T$	{ E.PLACE = newtemp(); gencode(+i, E ₁ .PLACE, T.PLACE, E.PLACE) }
(3)	$E \rightarrow T$	{ E.PLACE = newtemp(); E.PLACE = T.PLACE }
(4)	$T \rightarrow T_1 * F$	{ T.PLACE = newtemp(); gencode(*i, T ₁ .PLACE, F.PLACE, T.PLACE) }
(5)	$T \rightarrow F$	{ T.PLACE = newtemp(); T.PLACE = F.PLACE }
(6)	$F \rightarrow -F_1$	{ F.PLACE = newtemp(); gencode(@i, F.PLACE, _, F ₁ .PLACE) }
(7)	$F \rightarrow i$	{ F.PLACE = newtemp(); F.PLACE = ENTRYR(i) }
(8)	$F \rightarrow (E)$	{ F.PLACE = newtemp(); F.PLACE = E.PLACE }

例：赋值句 $A=B+C*(-D)$ 的自底向上分析

设：翻译此赋值句之前四元式的最大编号为K

$D) \#$
↑

$F_D \cdot PLACE$	F_D
@ ¹	—
(	(
* ¹	*
$T_c \cdot PLACE$	T_c
+ ¹	+
$E_B \cdot PLACE$	E_B
=	=
ENTRY(i_A)	i_A
—	#

前期全部移进

$) \#$
↑

$F'_D \cdot PLACE$	F'_D
(	(
* ¹	*
$T_c \cdot PLACE$	T_c
+ ¹	+
$E_B \cdot PLACE$	E_B
=	=
ENTRY(i_A)	i_A
—	#

归约— F_D 为 F'_D
产生四元式 $K+1$;

$(@^1, F_D \cdot PLACE, -, F'_D \cdot PLACE)$

$\#$
↑

)	)
$F'_D \cdot PLACE$	F'_D
(	(
* ¹	*
$T_c \cdot PLACE$	T_c
+ ¹	+
$E_B \cdot PLACE$	E_B
=	=
ENTRY(i_A)	i_A
—	#

移进)

$\#$
↑

$F''_D \cdot PLACE$	F''_D
* ¹	*
$T_c \cdot PLACE$	T_c
+ ¹	+
$E_B \cdot PLACE$	E_B
=	=
ENTRY(i_A)	i_A
—	#

归约(F'_D)为 F''_D

↑

$T' \cdot \text{PLACE}$	T'
$+^1$	$+$
$E_B \cdot \text{PLACE}$	E_B
$=$	$=$
$\text{ENTRY}(i_A)$	i_A
$-$	$\#$

归约 $T_c *^i F_D^*$ 为 T'
产生四元式 $K+2$

↑

$E \cdot \text{PLACE}$	E
$=$	$=$
$\text{ENTRY}(i_A)$	i_A
$-$	$\#$

归约 $E_B +^i T'$ 为 E
产生四元式 $K+3$

↑

$-$	

归约 $i_A = E$
产生四元式 $K+4$

$K+2: (*^i, T_c \cdot \text{PLACE}, F_D^* \cdot \text{PLACE}, T' \cdot \text{PLACE})$

$K+3: (+^1, E_B \cdot \text{PLACE}, T' \cdot \text{PLACE}, E \cdot \text{PLACE})$

$K+4: (: =, E \cdot \text{PLACE}, -, \text{ENTRY}(i_A))$

因此，四元式表中增加了4条四元式：

K	... (翻译此赋值语句之前的四元式)
$K+1$	$((@^i, F_D \cdot \text{PLACE}, _, F_D' \cdot \text{PLACE}))$
$K+2$	$((*^i, T_c \cdot \text{PLACE}, F_D'' \cdot \text{PLACE}, T' \cdot \text{PLACE}))$
$K+3$	$((+^i, E_B \cdot \text{PLACE}, T' \cdot \text{PLACE}, E \cdot \text{PLACE}))$
$K+4$	$((=, E \cdot \text{PLACE}, _, \text{ENTRY}(i_A)))$

5.4.3 布尔表达式的翻译

1. 布尔表达式的作用

- 用作控制语句中的条件表达式
- 用在逻辑赋值语句中

2. 布尔表达式的形式

- (1)单个布尔量;(2)布尔量的运算not, and, or, 优先级比关系运算低;(3)关系运算: $E1 \text{ rop } E2$, $E1E2$ 是算术表达式, rop为关系运算符: $>, <, >=, <=, =$

3. 布尔表达式的文法描述

(1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$

(2) $\langle BE \rangle \rightarrow \langle BT \rangle$

(3) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$

(4) $\langle BT \rangle \rightarrow \langle BF \rangle$

(5) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$

(6) $\langle BF \rangle \rightarrow (\langle BE \rangle)$

(7) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$

(8) $\langle BF \rangle \rightarrow i \text{ rop } i$

(9) $\langle BF \rangle \rightarrow i$

例如: $\text{not } a \text{ and } (y > z);$

4. 布尔量的代码结构 (作为控制语句的条件式)

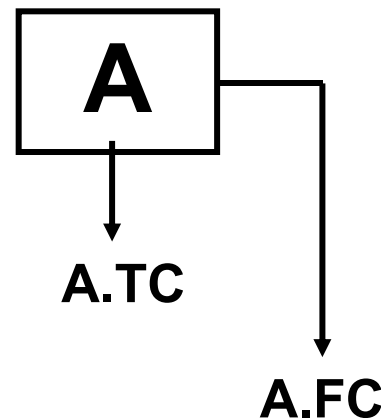
— 每个布尔量(布尔变量或布尔常量)的目标结构有两个出口，真出口指向为真时应跳转的位置；假出口指向为假时应跳转的位置。

• 布尔量翻译为两条四元式：

- (jnz, A, _, P) : 真出口，当A为真时跳转到四元式P
- (j, _, _, q) : 假出口，无条件跳转到四元式q

• 关系表达式也翻译为两条四元式：

- (jrop, i1, i2, P) : 真出口，当i1 rop i2为真时转四元式P
- (j, _, _, q) : 假出口，无条件跳转到四元式q



在翻译布尔表达式时，后面的语句未翻译，p,q如何知道？

• 解决方法是：回填技术

- 已知直接填入；未知先填0，等以后再填
- 若多个转移位置相同，但又不将这些未知且出口相同的四元
- 布尔表达式： $A \text{ and } B \text{ and } C > D$

当扫描到and时，对A进行归约，产生两个四元式1, 2，其中真出口已知，为3

```
(1) (jnz, A, —, 3)
(2) (j, —, —, 0)
(3) (jnz, B, —, 5)
(4) (j, —, —, 2)
(5) (j>, C, D, 0)
(6) (j, —, —, 4)
```

/* B的四元式 */

假出口未知，填入0

当扫描到B后的and时，对

假出口仍未知，但它与上两个布尔量的假出口相同，则链接起来，填4

$C > D$
两
时
0

生成了真、假出口两个链，两个0就是待填部分。等到以后翻译到后面再填入。

将 P_1, P_2 为链首两个四元式链合并在一起，可以用下述过程，返回合并后的四元式链首：

```
merge(P1, P2)
{
    if P2 == 0) return (P1);
    else {
        P := P2;
        while (四元式P的第4分量内容不为0)
        do
            P = 四元式P的第4分量内容;
            把P1填进四元式P的第4分量;
        return (P2);
    }
}
```

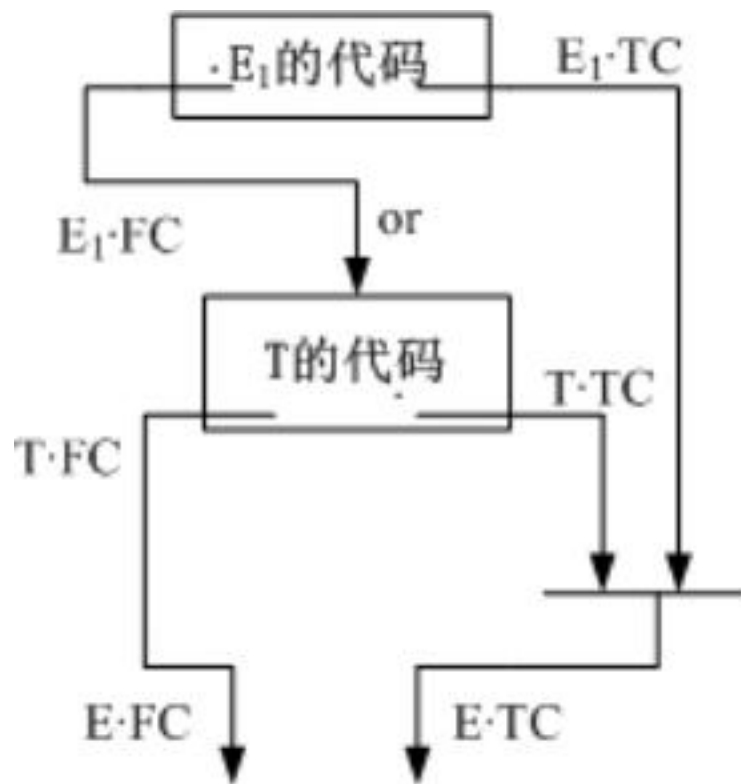
- 假出口链上的四元式应转向相同的位置，所以一旦知道转向的真实位置，就应返填，返填是将已知位置填入链上的所有四元式的第四个分量。
- 用backpatch，把已知位置t填入以P为链首的四元式中：

```
BACKPATCH(P, t)
{
    Q=P;
    while (Q!=0) do {
        m = 四元式Q的第4分量内容;
        把t填进四元式Q的第4分量;
        Q = m;
    }
}
```

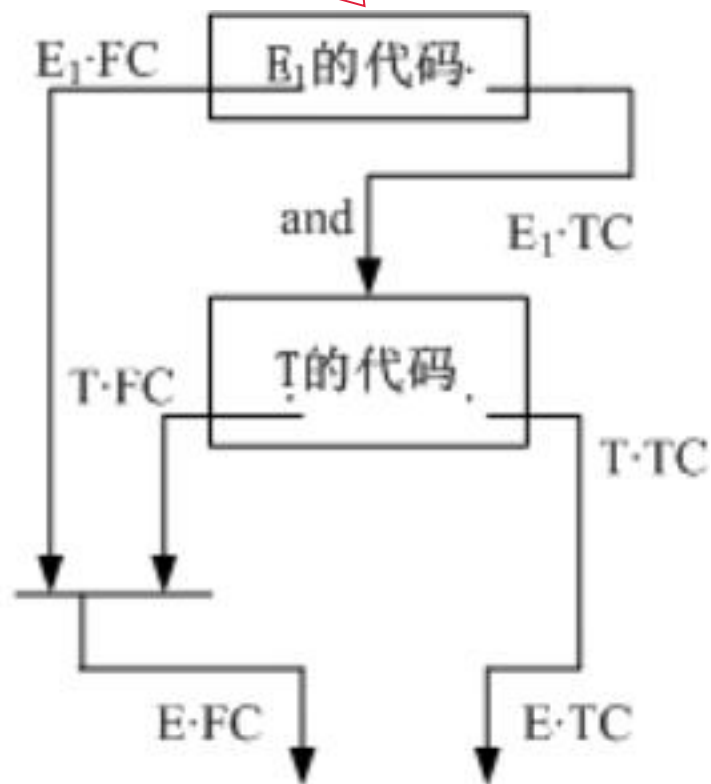
5. 布尔表达式的翻译

- 布尔表达式是带有not, and, or 的表达式
 - 第一种翻译方法可以象算术表达式一样翻译，先计算每个因子的值，再计算整个表达式的值。
 - 第二种翻译方法采取优化措施进行翻译。
-
- E1 or T 的优化：只要E为真，后面的表达式就不必计算，只有当E为假时才读取T。
 - E1 and T 的优化：只要E为假，后面的表达式就不必计算，只有当E为真时才读取T。

如果是翻译not E1, 则E的真假出口正好与E1相反



(a) $E_1 \text{ or } T$ 的目标结构



(b) $E_1 \text{ and } T$ 的目标结构

翻译布尔表达式时，当读到not，and，or时，要进行归约，因此应对文法进行改造

原因：

- 句柄归约在先，语义动作在后。
- 归约时，栈内句柄各符号的语义信息与该句柄同时出栈，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量同时入栈。
- 为了在某处调用语义动作，就必须先归约，因此，有时需要改写文法。

翻译布尔表达式时，当读到not, and, or时，要进行归约，因此应对文法进行改造，改造前后的文法为：

- | | |
|---|---|
| (1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$ | (6) $\langle BF \rangle \rightarrow (\langle BE \rangle)$ |
| (2) $\langle BE \rangle \rightarrow \langle BT \rangle$ | (7) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$ |
| (3) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$ | (8) $\langle BF \rangle \rightarrow i \text{ rop } i$ |
| (4) $\langle BT \rangle \rightarrow \langle BF \rangle$ | (9) $\langle BF \rangle \rightarrow i$ |
| (5) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$ | |
- 改造前的文法

- | | |
|---|---|
| (1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$ | (7) $\langle BF \rangle \rightarrow (\langle BE \rangle)$ |
| (2) $\langle BE \rangle \text{ or } \rightarrow \langle BE \rangle \text{ or }$ | (8) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$ |
| (3) $\langle BE \rangle \rightarrow \langle BT \rangle$ | (9) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$ |
| (4) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$ | (10) $\langle BF \rangle \rightarrow i \text{ rop } i$ |
| (5) $\langle BT \rangle \text{ and } \rightarrow \langle BT \rangle \text{ and }$ | (11) $\langle BF \rangle \rightarrow i$ |
| (6) $\langle BT \rangle \rightarrow \langle BF \rangle$ | |
- 改造后的文法

总结前面的分析，各产生式的翻译为：

NXQ表示当前产生的四元式的编号

编号	产生式	翻译
11)	$\langle BF \rangle \rightarrow i$	<pre> { <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jnz, entry(i), _, 0); gencode(j, _, _, 0) }</pre>
10)	$\langle BF \rangle \rightarrow i^{(1)} \text{ rop } i^{(2)}$ /*rop 为< , <= , >= , ==, < 或>*/	<pre> { <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jrop,i^(1).PLACE,i^(2).PLACE,0) gencode(j, _, _, 0) }</pre>
	$\langle BF \rangle \rightarrow \langle AE \rangle^{(1)} \text{ rop } \langle AE \rangle^{(2)}$ /*<AE>是算术表达式 */	<pre> { <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jrop,<AE>^(1).PLACE,<AE>^(2).PLACE, 0) gencode(j, _, _, 0) }</pre>
		<pre> { <BF>.TC=<BF>^(1).FC; <BF>.FC=<BF>^(1).TC }</pre>

Not逻辑运算



产生式6: $\langle BT \rangle \rightarrow \langle BF \rangle$,
3: $\langle BE \rangle \rightarrow \langle BT \rangle$ 的翻译与
7相似, 都是将右边的真假
出口直接赋值到左边

编号	产生式	语义规则
(7)	$\langle BF \rangle \rightarrow (\langle BE \rangle)$	$\{ \langle BF \rangle.TC = \langle BE \rangle.TC;$ $\langle BF \rangle.FC = \langle BE \rangle.FC \}$
(5)	$\langle BT \rangle_{and} \rightarrow \langle BT \rangle \text{ and }$	$\{ \text{backpatch}(\langle BT \rangle.TC, NXQ);$ $\langle BT \rangle_{and}.FC = \langle BT \rangle.FC;$
(4)	$\langle BT \rangle \rightarrow \langle BT \rangle_{and} \langle BF \rangle$	$\{ \langle BT \rangle.TC = \langle BT \rangle_{and}.TC;$ $\langle BT \rangle.FC = \text{merge}(\langle BT \rangle_{and}.FC, \langle BF \rangle.FC) \}$
(2)	$\langle BE \rangle_{or} \rightarrow \langle BE \rangle \text{ or }$	$\{ \text{backpatch}(\langle BE \rangle.FC, NXQ);$ $\langle BE \rangle_{or}.TC = \langle BE \rangle.TC;$
(1)	$\langle BE \rangle \rightarrow \langle BE \rangle_{or} \langle BT \rangle$	$\{ \langle BE \rangle.FC = \langle BE \rangle_{or}.FC;$ $\langle BE \rangle.TC = \text{merge}(\langle BE \rangle_{or}.TC, \langle BT \rangle.TC) \}$

(5)(4)构成and逻辑运算

(2)(1)构成or逻辑运算

- 布尔表达式: $A \text{ and } B \text{ and } C > D$ 的四元式为:

(1) (jnz, A, —, 3)	} /* A的四元式 */
(2) (j, —, —, 0)	
(3) (jnz, B, —, 5)	} /* B的四元式 */
(4) (j, —, —, 2)	
(5) (j>, C, D, 0)	} /* C>D的四元式 */
(6) (j, —, —, 4)	



5.4.4 控制语句的翻译

- 控制语句包括：
 - if 语句
 - While 语句
 - Dowhile语句
 - For 语句

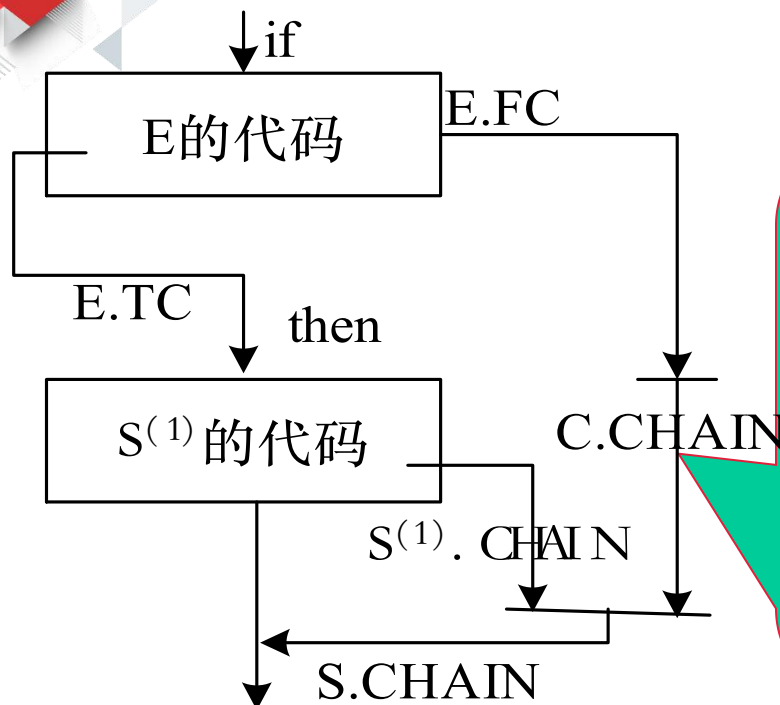
IF语句的翻译

1. IF语句的文法(S是开始符号)

- (1) $S \rightarrow C S(1)$
- (2) $C \rightarrow \text{if } E \text{ then}$
- (3) $S \rightarrow T S(2)$
- (4) $T \rightarrow C S(1) \text{ else}$

- 产生式(1),(2)生成无else 的IF语句结构
- 产生式(2),(3),(4)生成if — then — else 的语句结构

2. IF语句的目标结构及其翻译(无else的结构)



if $a > b$ then $x = 3$;

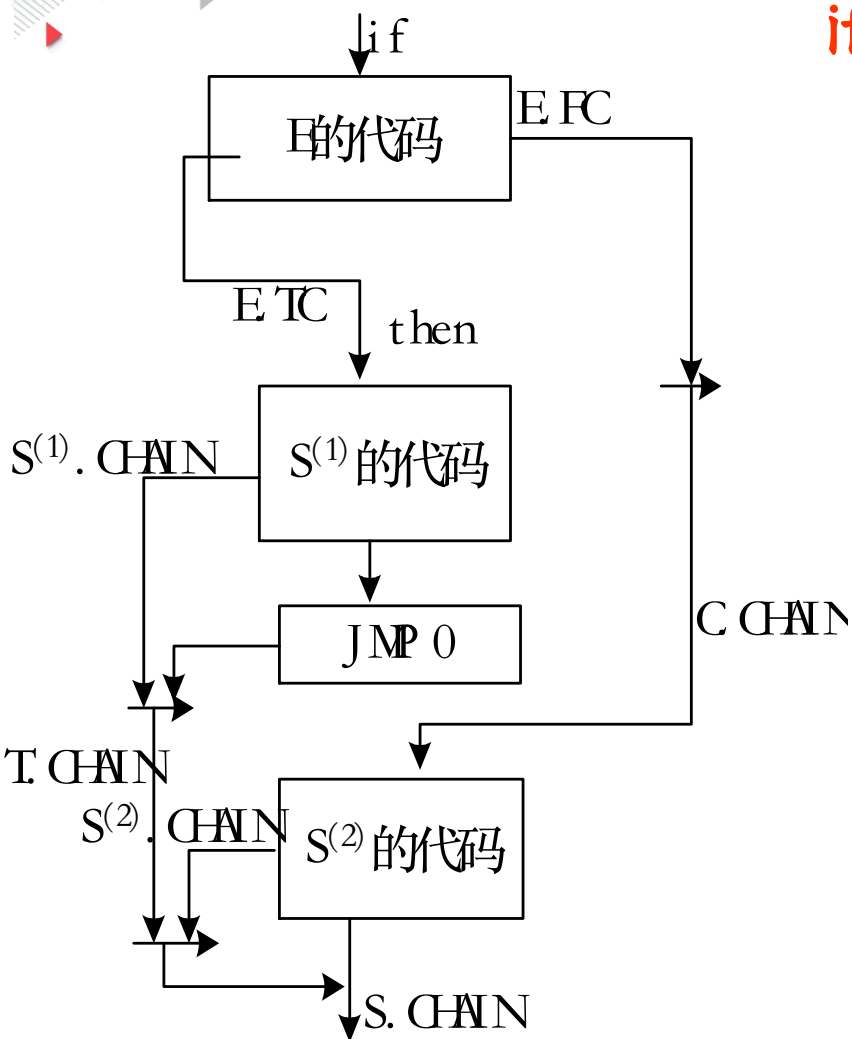
C.Chain的作用：由于在用第一个产生式进行归约时，只生成了条件式E的代码，扫描then时可以回填E.TC，E.FC还不能回填，必须向后传递到下一个产生式中需要通过语义变量C.Chain记录。

(1)	$S \rightarrow C S^{(1)}$	$\{ S \cdot CHAIN = MERG(C \cdot CHAIN, S^{(1)} \cdot CHAIN) \}$
(2)	$C \rightarrow \text{if } E \text{ then}$	$\{ \text{BACKPATCH}(E \cdot TC, NXQ);$ $C \cdot CHAIN = E \cdot FC \}$

2. IF语句的目标结构及其翻译

— 有else的结构

if $a > b$ then $x := 3$ else $x := 5$;



语义规则

$C \rightarrow \text{if } E \text{ then}$
 $\{ \text{backpatch}(E.TC, NXQ);$
 $C.CHAIN = E.FC \quad \}$

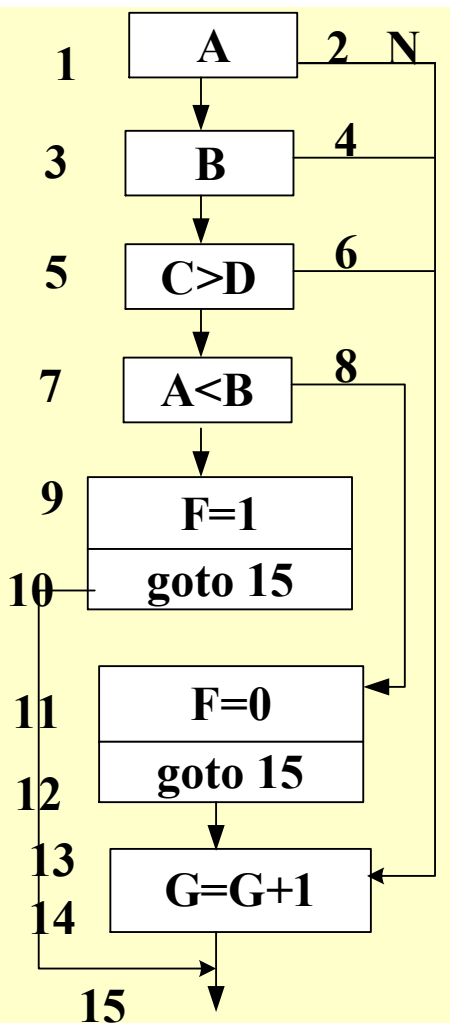
$S \rightarrow T S^{(2)}$
 $\{ S.CHAIN =$
 $\text{merge}(T.CHAIN, S^{(2)}.CHAIN) \}$

$T \rightarrow C S^{(1)} \text{ else}$
 $\{ q = NXQ;$
 $\text{gencode}(j, _, _, 0);$
 $\text{backpatch}(C.CHAIN, NXQ);$
 $T.CHAIN = \text{merge}(S^{(1)}.CHAIN, q) \}$

例：将下面的IF语句翻译为四元式序列



if A and B and (C>D) then
if A<B then F=1
else F=0
else G=G+1

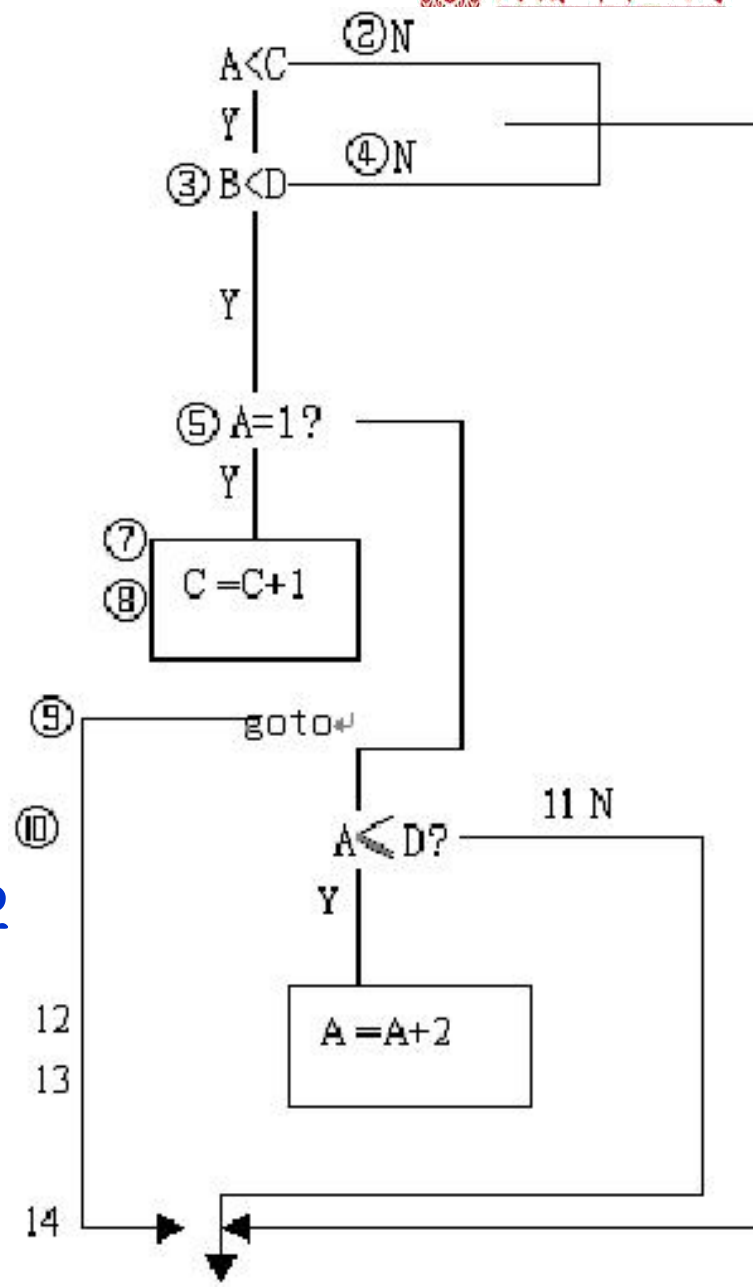


1. (jnz, A, _, 3) /*A的四元式*/
2. (j, _, _, 13)
3. (jnz, B, _, 5) /*B的四元式*/
4. (j, _, _, 13)
5. (j>, C, D, 7) /*C>D的四元式*/
6. (j, _, _, 13)
7. (j<, A, B, 9) /*A<B的四元式*/
8. (j, _, _, 11)
9. (=, 1, _, F) /*F :=1的四元式*/
10. (j, -, -, 15)
11. (=, 0, _, F) /*F :=0的四元式*/
12. (j, _, _, 15)
13. (+, G, 1, T) /*G:=G+1的四元式*/

练习：将下面的语句翻译为四元式序列

if (A<C) and (B<D) then
 if A=1 then
 C=C+1
 else if A≤D then
 A=A+2;

1.(j<,A,C,3)
 2.(j,-,-,14)
 3.(j<,B,D,5)
 4.(j,-,-,14)
 5.(j=,A,1,7)
 6.(j,-,-,10)
 7.(+,C,1,T₁)
 8.(=,T₁,-,C)
 9.(j,-,-,14)
 10.(j≤,A,D,12)
 11.(j,-,-,14)
 12.(+,A,2,T₂)
 13.(=,T₂,-,A)
 14.



二 Do while语句的翻译

• 1. 文法描述

$S \rightarrow \text{do } S^{(1)} \text{ while } E$

$D \rightarrow \text{do}$

$U \rightarrow DS^{(1)} \text{ while}$

$S \rightarrow UE$

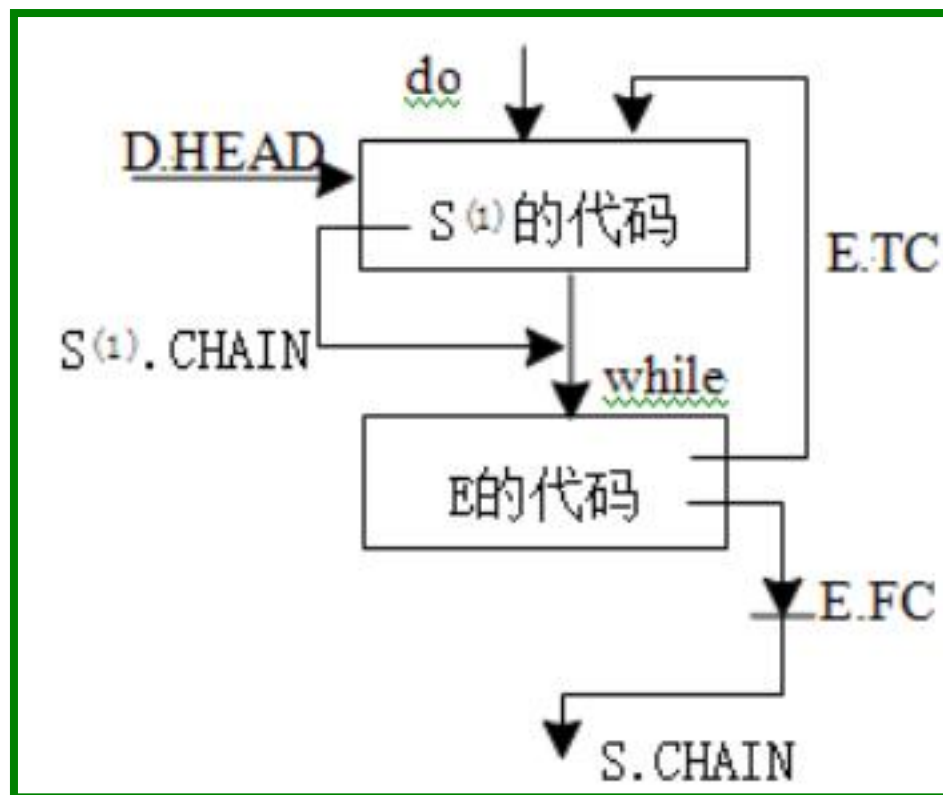
例:

do

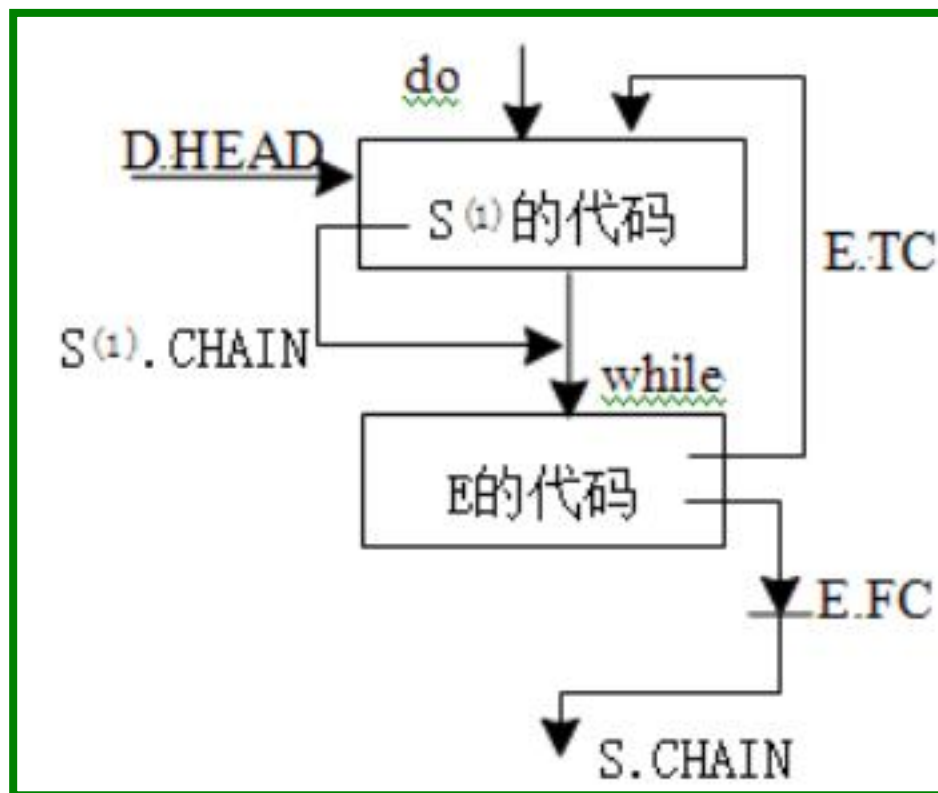
$x=3+a;$

while ($a>b$);

• 2. 目标结构



3. 翻

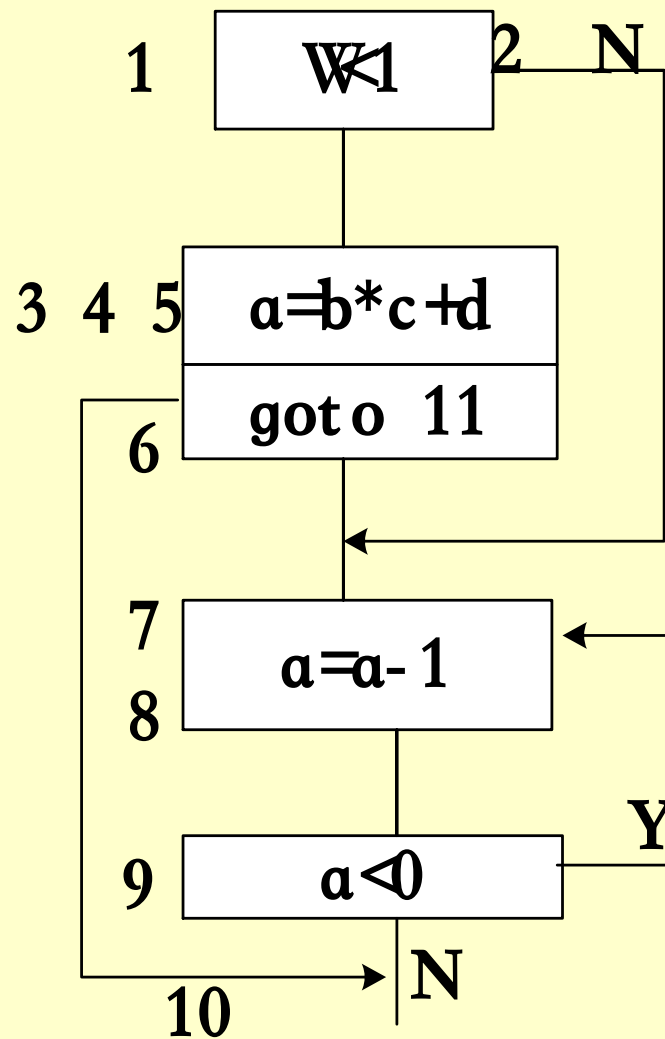


编号	产生式	语义规则
(1)	$D \rightarrow do$	{ $D.HEAD = NXQ$ }
(2)	$U \rightarrow DS^{(1)} while$	{ $U.HEAD = D.HEAD;$ $backpatch(S^{(1)}.CHAIN, NXQ)$ }
(3)	$S \rightarrow UE$	{ $backpatch(E.TC, U.HEAD);$ $S.CHAIN = E.FC$ }

将下面的语句翻译为四元式序列

- (1) $(j <, w, 1, 3)$
- (2) $(j, _, _, 7)$
- (3) $(*, b, c, T_1)$
- (4) $(+, T_1, d, T_2)$
- (5) $(=, T_2, _, a)$
- (6) $(j, _, _, 10)$
- (7) $(-, a, 1, T_3)$
- (8) $(=, T_3, _, a)$
- (9) $(j <, a, 0, 7)$
- (10)

if $w < 1$ then $a = b * c + d$
else do $a = a - 1$
while($a < 0$);



FOR语句的翻译

1. 文法描述

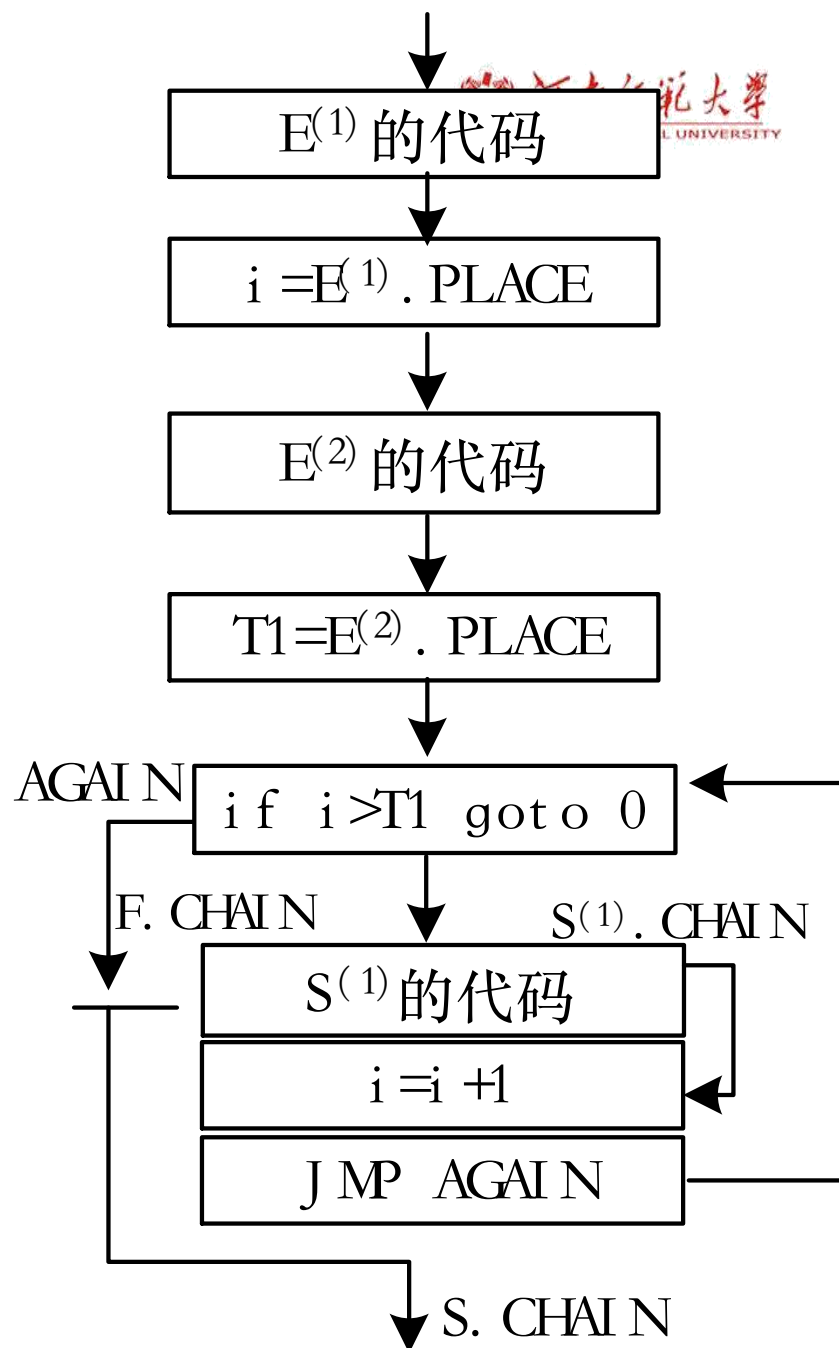
(1) $F \rightarrow \text{for } i = E^{(1)} \text{ to } E^{(2)} \text{ do}$

(2) $S \rightarrow FS^{(1)}$

例:

for i=1 to a+b do
x=x+1;

2. 目标结构



FOR语句的翻译

for i=1 to a+b do
x=x+1;



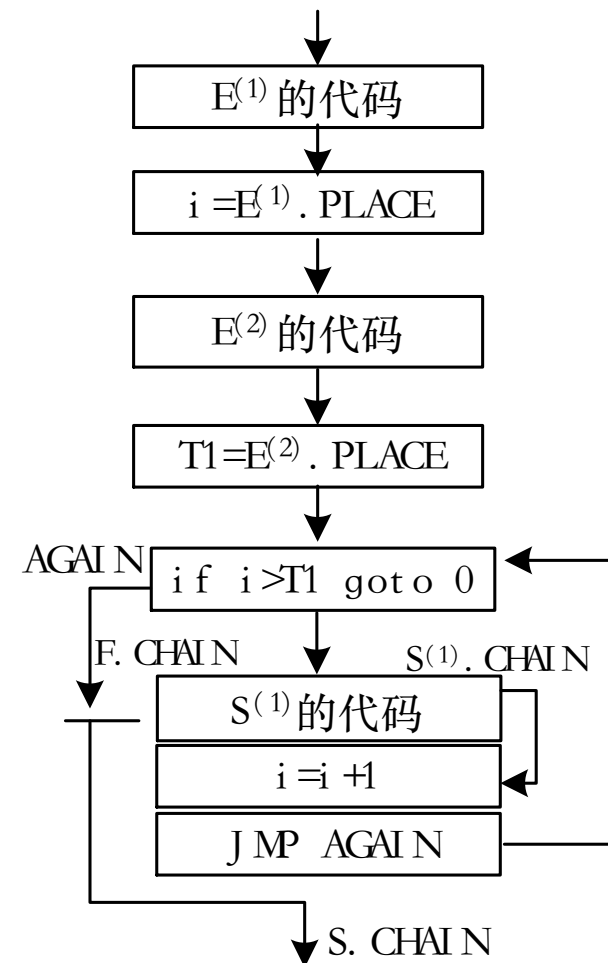
$F \rightarrow \text{for id} = E^{(1)} \text{ to } E^{(2)} \text{ do}$

```
{ F.PLACE = entry(id);  
  gencode(=, E(1).PLACE, _, F.PLACE);  
  T1 = newtemp(); /*T1用于存放循环终值*/  
  gencode(=, E(2).PLACE, _, T1);  
  F.AGAIN = NXQ;  
  F.CHAIN = NXQ;  
  gencode(j>, F.PLACE, T1, 0) }
```

$S \rightarrow F S^{(1)}$

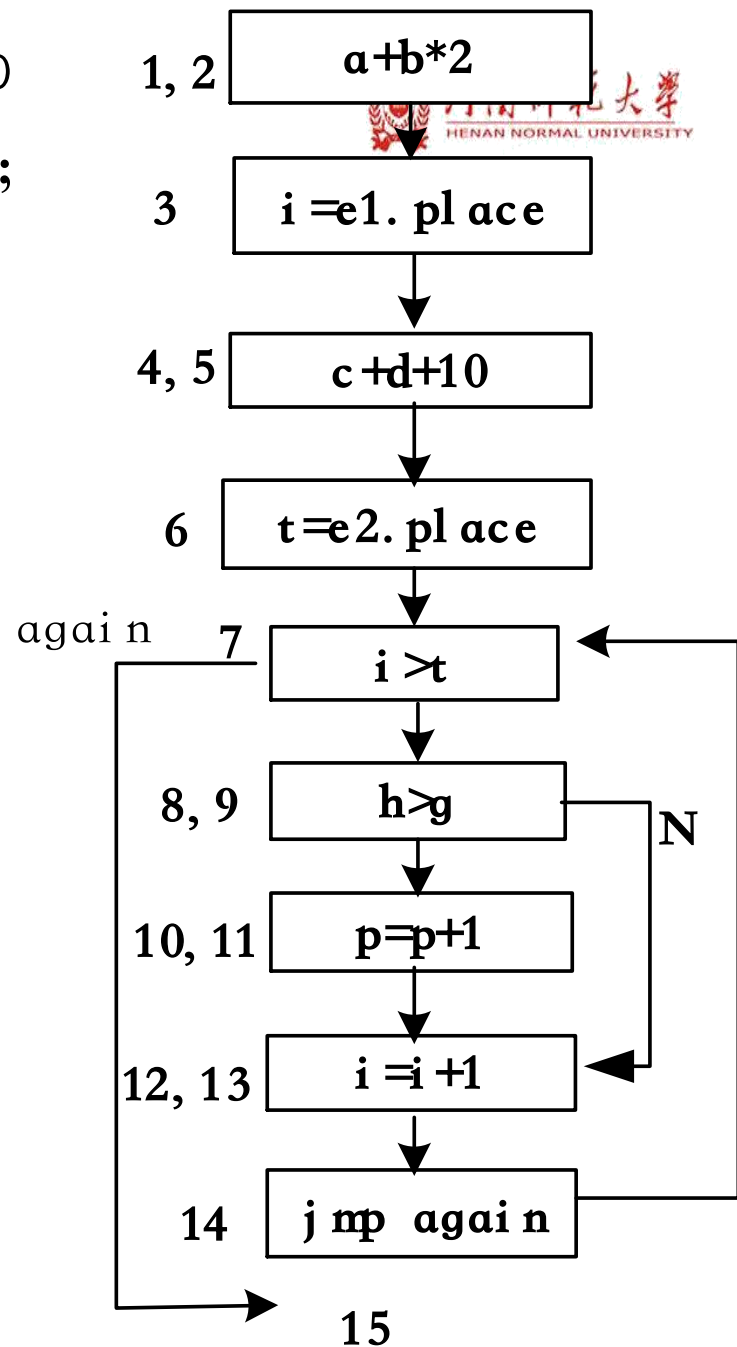
/*S是开始符号*/

```
{ backpatch(S(1).CHAIN, NXQ)  
  gencode(+, F.PLACE, 1, F.PLACE);  
  gencode(j, _, _, F.AGAIN);  
  S.CHAIN = F.CHAIN }
```



例: for $i = a+b*2$ to $c+d+10$ do
if $h>g$ then $p=p+1$;

- (1) $(*, b, 2, T_1)$
- (2) $(+, a, T_1, T_2)$
- (3) $(=, T_2, _, i)$
- (4) $(+, c, d, T_3)$
- (5) $(+, T_3, 10, T_4)$
- (6) $(=, T_4, _, t)$
- (7) $(j>, i, t, 15)$
- (8) $(j> h, g, 10)$
- (9) $(j, _, _, 12)$
- (10) $(+, p, 1, T_5)$
- (11) $(=, T_5, _, p)$
- (12) $(+, i, 1, T_6)$
- (13) $(=, T_6, _, i)$
- (14) $(j, _, _, 7)$
- (15)



5.5 中间代码生成器设计

- 语法制导的翻译实际上就是在语法分析的基础上，当分析完一个正确的语法单位后，调用相应的语义处理程序，直接生成相应的四元式表。
- 在第3章使用递归下降的分析方法进行了Sample语言的语法分析器
- 在此基础上添加语义处理

例：为IF语句添加语义处理

```
ifs( ) { /* If语句的语法分析*/  
    token = getnexttoken();  
    If(token!="if") error;  
    token= getnexttoken();  
    bexp();  
    token = getnexttoken();  
    If(token!="then") error;  
    token = getnexttoken();  
    ST_SORT();/*调用函数处理then后的可执行语句*/  
    token = getnexttoken();  
    If(token!= "else") error;  
    token = getnexttoken();  
    ST_SORT();/*处理else后的可执行语句*/  
}
```

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>

ifs() { /* 添加语义处理后的程序 */

token = getnexttoken();

if (token != "if") error;

token = getnexttoken();

(e.tc, e.fc) = bexp();

token = getnexttoken();

if (token != "then") error("缺then");

backpatch(e.tc, nxq); /*回填真出口e.tc*/

token = getnexttoken();

s1.chain = ST_SORT(token); /*处理s1,返回s1链*/

token = getnexttoken();

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>

```
if token="else" /* 处理else部分*/
```

```
{ q = nxq;  
  gencode(j, —, —, 0);  
  backpatch(e.fc, nxq); /*回填假出口e.fc*/  
  t.chain = merg(s1.chain, q);  
  getnexttoken(token);  
  s2.chain = ST_SORT(token);  
  s.chain = merg(t.chain, s2.chain);  
  return(s.chain)
```

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>

```
}  
else /* if -- then结构时*/
```

```
  return(merg(s1.chain, e.fc))
```

```
}
```

Sample语言语法制导翻译程序的总控程序的编写

- 语法制导翻译程序的总控程序既要负责处理调用各个语句的处理，又要负责翻译PROGRAM的产生式（ $\langle \text{程序} \rangle ::= \text{program } \langle \text{标识符} \rangle ; \langle \text{分程序} \rangle$ 和 $\langle \text{分程序} \rangle ::= \langle \text{常量说明} \rangle \langle \text{变量说明} \rangle \langle \text{复合句} \rangle .$ ）。
- 当匹配了” program”和程序名标识符id后，应生成四元式（program, id, $_$, $_$ ），这个四元式表示目标程序执行前的一些准备工作，这些工作通常和机器有关，包括包护一些寄存器，设置保护区，为用户程序分配一定的运行空间等。
- 当匹配到” end.”这两个符号时，则应生成四元式（sys, $_$, $_$, $_$ ），用于表示恢复寄存器，释放保留区，退出程序的运行状态等操作。

```
void paser_senmatic( ) /*语法制导翻译的总控程序*/
{
    token = getnexttoken();
    if (token != "program" ) error(); /*程序中缺少program*/
    token = getnexttoken();
    if (token 不是 标识符) error(); /*program后应跟程序名称*/
    gencode("program",token,_,_);

    .....
    token = getnexttoken();
    if (token 不是 'end.') error(); /*end.标识整个程序结束*/
    gencode("sys",_,_,_);
}
```

用C语言实现repeat()

```
int *repeats()/* repeat语句的语法制导的翻译过程*/
{
    r_head = nxq; /*记录repeat语句开头的四元式编号*/
    token= getnexttoken();/*读取下一个token字*/
    sl_chain = ST_SORT(token);/*处理repeat后的语句*/
    token= getnexttoken();
    if (!strcmp(token, ";")) /*读到;*/
    token= getnexttoken();
    if (strcmp(token, "until")) error("缺少 until");/*应为until*/
    backpatch(&sl_chain, &nxq);
    token= getnexttoken();
    bexp(&be_tc, &be_fc, token);/*处理布尔表达式*/
    backpatch(&be_fc, &r_head);/*回填假出口*/
    return &be_tc; /*返回真出口，即repeat语句的下一个语句的四元式编号*/
}
```

6. 解答:

$$(a+b)+(c+d/(e-3))*8$$

7. 解答:

(1) $ab-c+*$

(2) A not C D not or not or

(3)

$$abcde/+*+$$

(4) A B and C not D or or

(5) $a-bc-d+*+$

(6) A B or C D not E and or and



三元式	四元式
① $(+, a, b)$	1. $(+, a, b, T_1)$
② $(-, 1, -)$	2. $(-, T, -, T_2)$
③ $(+, c, d)$	3. $(+, c, d, T_3)$
④ $(*, 2, 3)$	4. $(*, T_2, T_3, T_4)$
⑤ $(+, a, b)$	5. $(+, a, b, T_5)$
⑥ $(+, c, 5)$	6. $(+, T_5, c, T_6)$
⑦ $(-, 4, 6)$	7. $(-, T_4, T_6, T_7)$